

# LABORATORIO DE RUTAS



Byron Calderón

[adrianbyron2005@gmail.com](mailto:adrianbyron2005@gmail.com)

---

# Laboratorio de rutas y algoritmos de búsqueda

## 1. Objetivo del proyecto

El objetivo de este proyecto es crear una página web para probar y comparar diferentes algoritmos de búsqueda de rutas y ver gráficamente cómo funcionan.

La aplicación usa una cuadrícula de 20 x 20 casillas, donde se puede poner un punto de inicio, una o varias metas, obstáculos y casillas con diferentes costes.

Los algoritmos que se pueden probar son:

- BFS
- DFS
- Búsqueda de Coste Uniforme (UCS)
- A\*

La página muestra cómo explora cada algoritmo la cuadrícula y la ruta que encuentra. También ayuda a comparar los cuatro algoritmos usando el mismo mapa.

## 2. Tecnologías utilizadas

Para hacer el proyecto se han utilizado:

- HTML
- CSS
- JavaScript

## 3. Estructura del proyecto

El proyecto está organizado dentro de la carpeta **juego-laberinto**:

**juego-laberinto/** : **index.html**, **style.css**, **algoritmos.js**, **app.js**

- **index.html**: contiene la estructura, controles, botones, cuadrícula y resultados.
- **style.css**: contiene los estilos, tamaños, colores, botones y estados de las casillas.
- **algoritmos.js**: contiene BFS, DFS, UCS, A\* y las funciones necesarias para realizar las búsquedas.
- **app.js**: controla la cuadrícula, las interacciones, la ejecución, las animaciones, los escenarios y la comparación.

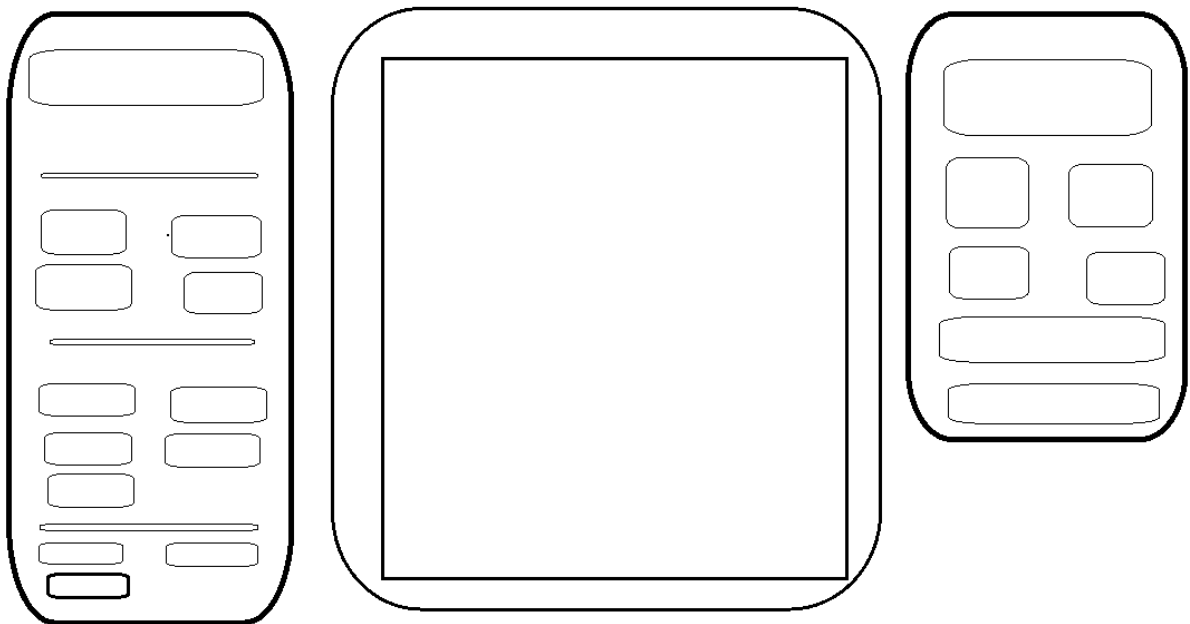
## 4. Cómo se construyó el proyecto

### Paso 1. Pensar la estructura de la página

Antes de empezar con el código se hizo una idea de cómo quería que fuera la página.

Para explicarle mejor la idea a la IA hice un pequeño dibujo en Paint, donde le mostré cómo quería organizar más o menos la página.

La idea principal era tener un encabezado, los controles y debajo la cuadrícula de 20 x 20.



### Paso 2. Crear el HTML y el CSS

Primero empecé haciendo HTML junto con el CSS.

No hice toda la página de golpe, sino que fui haciéndola poco a poco.

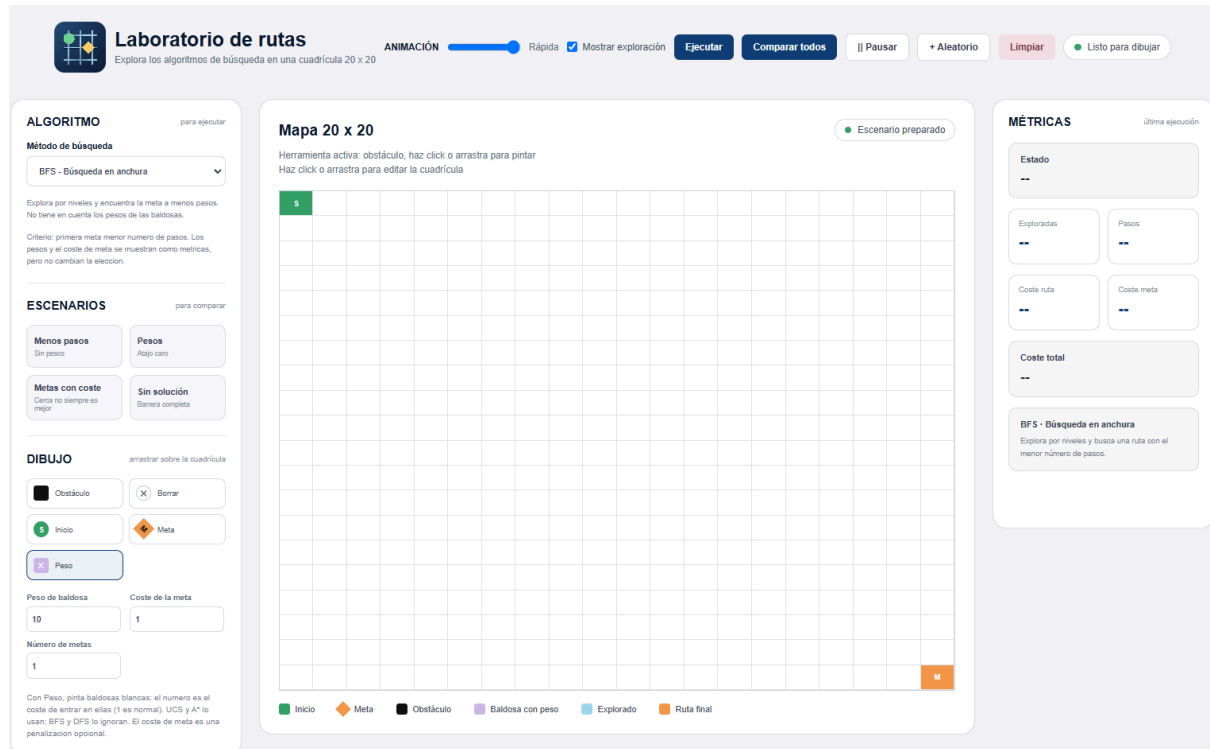
Primero trabajé en el header, colocando el título, el texto y la estructura que quería.

Después pasé al body, donde empecé a poner la parte de la cuadrícula y los diferentes controles.

Para que la página quedará exactamente como yo quería, fui explicándole a la IA cómo quería cada parte y utilizando el dibujo de Paint como una referencia(no es un dibujo perfecto).

Después trabajé en la cuadrícula de 20 x 20 y se le pidió a la IA que me genere una cuadrícula 20 x 20, que esté bien proporcionado respecto a los demás cuadrados y que se vea limpio.

También se fueron ajustando los tamaños y espacios de diferentes partes de la página.



## Paso 3. JavaScript

Después se empezó con JavaScript.

Primero le pedí a la IA que la cuadrícula pudiera interactuar con el usuario y que añada estas opciones:

- inicio
- metas
- obstáculos
- pesos
- borrar casillas

Después se añadió la posibilidad de tener varias metas.

## Paso 4. Algoritmos

Después se añadieron:

- BFS
- DFS
- UCS
- A\*.

## Paso 5. Costes y resultados

Después se añadió el sistema de pesos.

Una casilla normal tiene un coste de 1, pero si ponemos más números de peso, la casilla tendrá ese peso en donde lo seleccionamos.

Añadí un coste para las metas y esto permite hacer que una meta esté más cerca pero tiene un coste mayor y así se pueden comparar los algoritmos por el coste total y no solo por los pasos.

## Paso 6. Correcciones

Una de las partes que tuve que ir corrigiendo fue que se pueda visualizar la búsqueda o sea que al ejecutar un algoritmo quería que se vieran las casillas que iba explorando de un color azul suave, pero al principio no aparecían como yo quería.

Se fueron haciendo correcciones hasta conseguir que las casillas exploradas se vean mientras se ejecuta.

The screenshot displays a web-based pathfinding application interface. It is divided into three main sections: **ALGORITMO** (Algorithm), **Mapa 20 x 20** (Map), and **MÉTRICAS** (Metrics).

- ALGORITMO:** Contains a dropdown menu set to "BFS - Búsqueda en anchura". Below it, there is a description of the algorithm and a section for scenarios with buttons like "Menos pasos", "Pesos", "Metas con coste", and "Sin solución". At the bottom, there is a "DIBUJO" section with icons for obstacles, start, goal, and weight, along with input fields for "Peso de baldosa" (1) and "Coste de la meta" (1).
- Mapa 20 x 20:** A 20x20 grid where the search path is visualized. The start cell (S) is at the top left, and the goal cell (M) is at the bottom right. A series of light blue cells form a path from S to M. A status bar at the top right of the grid indicates "Explorando con BFS - Búsqueda en anchura".
- MÉTRICAS:** A sidebar on the right showing the current state. It includes a "Estado" section with "Explorando con BFS - Búsqueda en anchura". Below this, there are boxes for "Exploradas" (132), "Pasos" (--), "Coste ruta" (--), "Coste meta" (--), and "Coste total" (--). At the bottom, it lists the active algorithm: "BFS - Búsqueda en anchura" with a brief description.

Una de las partes que más fui ajustando fue el tamaño y algunas proporciones de la página, para que la cuadrícula y los controles estén bien colocados.

También se hicieron cambios para que la aplicación pudiera trabajar con varias metas y con diferentes costes.

## Paso 7. Escenarios y pruebas

Se hizo cuatro escenarios para poder comprobar las diferencias entre los algoritmos, escenarios como: **Menos pasos**, **Pesos**, **Metas con coste**, **Sin solución**.

# 5. Arquitectura y funcionamiento

El funcionamiento básico es:

**Usuario**



**Modifica la cuadrícula**



**Pone inicio, metas, obstáculos y pesos**



**Selecciona un algoritmo**



**Pulsa "Ejecutar"**



**JavaScript ejecuta el algoritmo**



**Se muestran las casillas de búsqueda en celeste suave**



**Se encuentra una meta**



**Se reconstruye la ruta**



**Se muestran los pasos y costes**

La cuadrícula se representa con filas y columnas, cada casilla guarda su posición y el estado, por ejemplo inicio, meta, obstáculo, casilla normal o casilla con peso.

También existe el botón "Comparar todos", para comparar los cuatro algoritmos que se usaron que ya mostraré más adelante.

## 6. Algoritmos utilizados

| Algoritmo | Estructura               | Criterio de selección | Visitados | Límite                         |
|-----------|--------------------------|-----------------------|-----------|--------------------------------|
| BFS       | Cola                     | Menor número de pasos | Sí        | No tiene en cuenta los pesos   |
| DFS       | Pila                     | Profundidad           | Sí        | No garantiza la ruta más corta |
| UCS       | Lista ordenada por coste | Menor coste acumulado | Sí        | Puede explorar muchas casillas |
| A*        | Lista ordenada por f     | $g + h$               | Sí        | Depende de la heurística       |

### BFS

Busca por niveles y utiliza una cola y en mapas sin pesos encuentra una ruta con el menor número de pasos.

### DFS

Utiliza una pila y avanza por profundidad.

Puede encontrar una ruta pero se puede equivocar y tener que regresar todo el camino de vuelta hasta encontrar la meta.

## UCS

Selecciona la casilla que tiene el menor coste acumulado desde el inicio y tiene en cuenta los pesos de las casillas y busca una ruta con coste menor.

## A\*

Junta el coste acumulado con una estimación del coste que nos falta:

- $f = g + h$
- **g**: coste acumulado.
- **h**: estimación del coste que falta.
- **f**: valor utilizado para elegir qué explorar.

Se utiliza la distancia Manhattan porque solo se permiten movimientos horizontales y verticales.

## Binary Search

La búsqueda binaria encuentra algo cortando una lista a la mitad, una y otra vez, descartando la parte donde sabe que no está.

No se usa para calcular rutas aquí porque eso necesita moverse por un mapa de cuadrículas y caminos, no por una simple lista.

## 7. Reconstrucción de la ruta

Mientras hace la búsqueda, se guarda información sobre la casilla anterior.

Cuando se encuentra una meta, se vuelve desde la meta hasta el inicio utilizando esa información, después se muestra la ruta final en la cuadrícula.

## 8. Los cuatro escenarios

### Escenario 1. Menos pasos

No se utilizan pesos y se ponen obstáculos para crear diferentes caminos.

Y sirve para comprobar la búsqueda con diferente número de pasos.

ALGORITMO

para ejecutar

Método de búsqueda

BFS - Búsqueda en anchura

Explora por niveles y encuentra la meta a menos pasos. No tiene en cuenta los pesos de las baldosas.

Criterio: primera meta menor numero de pasos. Los pesos y el coste de meta se muestran como métricas, pero no cambian la elección.

ESCENARIOS

para comparar

Menos pasos

Sin pesos

Pesos

Atajo cero

Metas con coste

Cerca no siempre es mejor

Sin solución

Barrera completa

DIBUJO

arrastrar sobre la cuadrícula

Obstáculo

Borrar

Inicio

Meta

Peso

Peso de baldosa

Coste de la meta

Mapa 20 x 20

Herramienta activa: obstáculo, haz click o arrastra para pintar  
Haz click o arrastra para editar la cuadrícula

S

MÉTRICAS

última ejecución

Estado

Escenario: Menos pasos

Exploradas

364

Pasos

38

Coste ruta

38

Coste meta

1

Coste total

39

BFS - Búsqueda en anchura

Explora por niveles y busca una ruta con el menor número de pasos.

## Escenario 2. Atajo caro

Hay un camino corto con casillas de coste alto y otro camino más largo con menor coste.

Sirve para comparar pasos y coste total.

ALGORITMO

para ejecutar

Método de búsqueda

BFS - Búsqueda en anchura

Explora por niveles y encuentra la meta a menos pasos. No tiene en cuenta los pesos de las baldosas.

Criterio: primera meta menor numero de pasos. Los pesos y el coste de meta se muestran como métricas, pero no cambian la elección.

ESCENARIOS

para comparar

Menos pasos

Sin pesos

Metas con coste

Cercá no siempre es mejor

DIBUJO

arrastrar sobre la cuadrícula

Mapa 20 x 20

Herramienta activa: obstáculo, haz click o arrastra para pintar

Haz click o arrastra para editar la cuadrícula

S

+10

+10

+10

+10

+10

+10

+10

+10

+10

+10

+10

+10

+10

+10

+10

+10

M

MÉTRICAS

última ejecución

Estado

Escenario: Atajo caro

Exploradas

--

Pasos

--

Coste ruta

--

Coste meta

--

Coste total

--

BFS - Búsqueda en anchura

Explora las casillas por niveles y busca una ruta con el menor número de pasos.

## Escenario 3. Varias metas

Hay varias metas. Una está más cerca pero tiene un coste mayor, mientras que otra está más lejos y tiene un coste menor. Sirve para comprobar cómo se tienen en cuenta los pasos y los costes.

**ALGORITMO**para ejecutar

**Método de búsqueda**

BFS - Búsqueda en anchura

Explora por niveles y encuentra la meta a menos pasos. No tiene en cuenta los pesos de las baldosas.

Criterio: primera meta menor numero de pasos. Los pesos y el coste de meta se muestran como metricas, pero no cambian la eleccion.

**ESCENARIOS**para comparar

Menos pasos  
Sin pesos

Pesos  
Atajo caro

Metas con coste  
Cerca no siempre es mejor

Sin solución  
Barrera completa

**DIBUJO**arrastrar sobre la cuadrícula

Obstáculo

Borrar

Inicio

Meta

Peso

Peso de baldosa

Coste de la meta

1

1

Número de metas

1

**Mapa 20 x 20**

Herramienta activa: obstáculo, haz click o arrastra para pintar  
Haz click o arrastra para editar la cuadrícula

S

M

M

**MÉTRICAS**última ejecución

Estado

Escenario: Metas con coste

Exploradas

Pasos

Coste ruta

Coste meta

Coste total

BFS - Búsqueda en anchura

Explora las casillas por niveles y busca una ruta con el menor número de pasos.

## Escenario 4. Sin solución

Se coloca una barrera que impide llegar desde el inicio hasta la meta. Los cuatro algoritmos deben indicar que no existe una ruta.

**ALGORITMO**para ejecutar

**Método de búsqueda**

BFS - Búsqueda en anchura

Explora por niveles y encuentra la meta a menos pasos. No tiene en cuenta los pesos de las baldosas.

Criterio: primera meta menor numero de pasos. Los pesos y el coste de meta se muestran como metricas, pero no cambian la eleccion.

**ESCENARIOS**para comparar

Menos pasos  
Sin pesos

Pesos  
Atajo caro

Metas con coste  
Cerca no siempre es mejor

Sin solución  
Barrera completa

**DIBUJO**arrastrar sobre la cuadrícula

Obstáculo

Borrar

**Mapa 20 x 20**

Herramienta activa: obstáculo, haz click o arrastra para pintar  
Haz click o arrastra para editar la cuadrícula

S

Sin solución

**MÉTRICAS**última ejecución

Estado

Escenario: Sin solución

Exploradas

Pasos

Coste ruta

Coste meta

Coste total

BFS - Búsqueda en anchura

Explora por niveles y busca una ruta con el menor número de pasos.

## 9. Comparación de los algoritmos

La aplicación tiene un botón "**Comparar todos**".

Este botón permite ejecutar los cuatro algoritmos sobre el mismo escenario.

Se comparan:

- pasos
- coste total
- casillas exploradas
- resultado

| Escenario    | BFS                  | DFS                   | Coste uniforme      | A*                  |
|--------------|----------------------|-----------------------|---------------------|---------------------|
| Menos pasos  | 38 pasos / 39 coste  | 70 pasos / 71 coste   | 38 pasos / 39 coste | 38 pasos / 39 coste |
| Atajo caro   | 19 pasos / 182 coste | 193 pasos / 194 coste | 23 pasos / 24 coste | 23 pasos / 24 coste |
| Varias metas | 5 pasos / 55 coste   | 190 pasos / 191 coste | 38 pasos / 39 coste | 5 pasos / 55 coste  |
| Sin solución | Sin ruta             | Sin ruta              | Sin ruta            | Sin ruta            |

### Resultados

**Menos pasos:** BFS, Coste Uniforme y A\* encuentran una ruta de 38 pasos, mientras que DFS encuentra una ruta más larga de 70 pasos.

**Atajo caro:** BFS encuentra el camino con menos pasos, pero tiene un coste mucho mayor. Coste Uniforme y A\* encuentran un camino más largo, pero con un coste mucho menor.

**Varias metas:** BFS y A\* llegan a la meta que está cerca en 5 pasos, pero con un coste total de 55. Coste Uniforme elige la meta que está lejos porque consigue un coste total menor, 39.

**Sin solución:** los cuatro algoritmos indican que no existe una ruta.

Los resultados dicen que los algoritmos no buscan lo mismo. Aquí una muestra de la comparación con el escenario de Menos datos.

#### Comparación de algoritmos

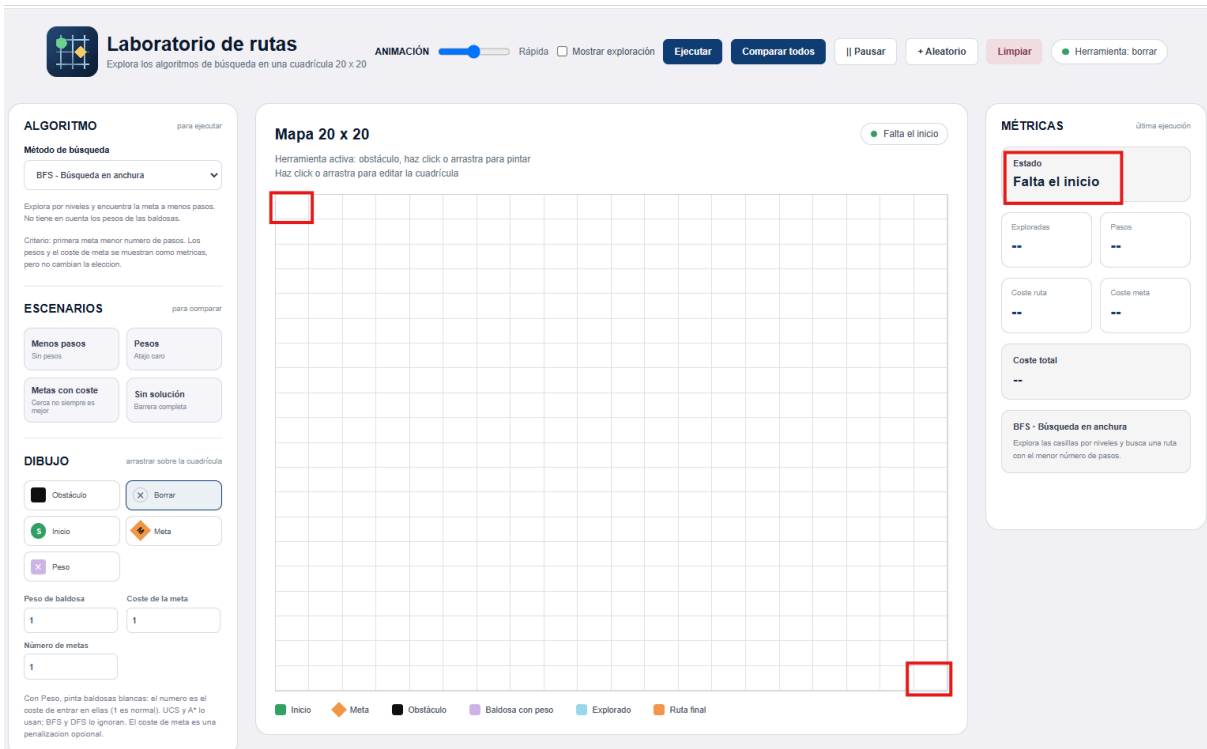
|                       |           |           |                 |                                        |
|-----------------------|-----------|-----------|-----------------|----------------------------------------|
| <b>BFS</b>            | Pasos: 38 | Coste: 39 | Exploradas: 364 | Busca por niveles y prioriza los pasos |
| <b>DFS</b>            | Pasos: 70 | Coste: 71 | Exploradas: 315 | Sigue una rama antes de probar otras   |
| <b>Coste uniforme</b> | Pasos: 38 | Coste: 39 | Exploradas: 364 | Prioriza el menor coste acumulado      |
| <b>A*</b>             | Pasos: 38 | Coste: 39 | Exploradas: 364 | Usa coste + estimación para buscar     |

## 10. Pruebas realizadas

| Prueba                     | Resultado esperado                  | Resultado obtenido                             |
|----------------------------|-------------------------------------|------------------------------------------------|
| Ruta sencilla              | Encontrar una ruta                  | Se encuentra una ruta                          |
| Sin inicio o meta          | Mostrar un aviso                    | Se muestra el aviso                            |
| Varias metas               | Trabajar con varias metas           | Funciona correctamente                         |
| Casillas con peso          | Tener en cuenta los costes          | Se tienen en cuenta                            |
| Obstáculos                 | Buscar otra ruta si existe          | Encuentra una ruta alternativa                 |
| Sin solución               | Indicar que no existe ruta          | Muestra que no hay solución                    |
| Editar después de ejecutar | Poder modificar y volver a ejecutar | Se puede volver a utilizar                     |
| UCS y A*                   | Coincidir en el coste óptimo        | Se comprobó en los escenarios correspondientes |

# 10.1. Prueba de inicio

Se realizó una prueba si no hay inicio ni una meta, lo cual aparecerá un aviso diciendo que falta el inicio



# 11. HTML, CSS y JavaScript

Cada tecnología tiene una función diferente.

| Tecnología | Función                                                                                                                                |
|------------|----------------------------------------------------------------------------------------------------------------------------------------|
| HTML       | Crear la estructura de la página, los textos, botones, controles, cuadrícula y resultados.                                             |
| CSS        | Dar diseño, define tamaños, espacios, bordes, botones y estados de las casillas.                                                       |
| JavaScript | Programar las funciones, los algoritmos, controla los clics, la cuadrícula, los costes, la animación, los resultados y la comparación. |

## 12. Uso de inteligencia artificial

Durante el proyecto utilicé un asistente de inteligencia artificial como ayuda para entender BFS, DFS, UCS y A\*, organizar partes de la página, partes del código y corregir problemas.

La página no se hizo todo de una vez, fui trabajando poco a poco. Lo que hice fue explicarle a la IA cómo quería que fuera la página y usé un dibujo de Paint para mostrarle más o menos lo que yo buscaba.

Después fui trabajando por partes, hace tiempo me enseñaron a organizarme en lo que es la creación de páginas, así que mis pasos fueron estos:

1. primero el header,
2. después el body y la cuadrícula,
3. después los estilos de CSS,
4. después JavaScript,
5. después los algoritmos,
6. y las pruebas, correcciones, etc.

Durante el proceso fui indicando cambios cuando algo no quedaba como quería, como por ejemplo cuando se ejecuta la búsqueda y aparecen los cuadraditos en azul, antes no me lo mostraba pero con las correcciones que tomé, al final funcionó como yo quería.

Mis peticiones a la IA son más o menos siempre el mismo, le digo mi objetivo y le digo que sea súper entendible, directo, y simple, así poder entender de la mejor manera las soluciones.

mira estoy en clases de IA, me pidieron hacer una pagina parecida al de mi profe, el archivo es la capreta juegos, dentro un css, html, y 2 js, uno para los algoritmos programados, y el otro para otro la lógica de la app sabes, quiero que sea la estructura igual, nsoe con que quieres empezar primero per te digo que la pagina tiene que ser fondo un poco gris tirando mas a blanco, la cabeza osea lo de arriba tienen que tener a la izquierda de todo que diga Laboratorio de rutas en negrilla por asi decirlo de color azul muy oscuro, bajo esa misma palabra que diga explora los algoritmos de busqueda en una cuadrícula 20 x 20(eso en letra mas pequeña ), a la izquierda de laboratorio de rutas una foto cuadrado con borders circulares y que la foto ya lo pondre despues nose pero bueno, desues a la derecha de LABORATORIO DE RUTAS UNA PALARA QUE DIGA ANIMACION EN MAYUSCULAS PERO MAS PEQUEÑO QUE LABORATORIO DE RUTAS PERO CON EL MISMO COLOR, y a la derecha de animacion una barra cn puntito circular que se pueda mover derecha a izquierda para indicar la rapidez nose, a la derecha una palabra que diga "rapida" y a la derecha de eso un cuadradito de check a la derecha Mostrar exploracion a la derecha boton de ejecutar que sea azul y que diga Ejecutar, y a la derecha comparar todos y a la derecha boton banco masomenos ahora que diga Pausar, a la derecha + Aleatorio a la derecha boton mas rosadito leve que diga limpiar y a la derecha uno que diga listo para jugar y botonsito verde chiquito dentro sabes, priemro eso por ahora, despues te digo lo demas, el css tambein eeh por cierto, DAME EL IDJEX.HTML Y EL CSS

## 12.1 Ejemplos de consultas representativas:

- "Explícame de forma sencilla cómo funcionan BFS, DFS, UCS y A\*."
- "Ayúdame a crear una cuadrícula de 20 x 20 con HTML, CSS y JavaScript."
- "Las casillas exploradas no aparecen durante la búsqueda, ayúdame a corregirlo."

## 13. Limitaciones

- La cuadrícula tiene un tamaño solo de 20 x 20.
- Solo se pueden hacer movimientos horizontales y verticales.
- El proyecto no usa ninguna base de datos.

## 14. Posibles mejoras

Algunas mejoras que podría añadir en el futuro son:

- utilizar mapas más grandes.
- añadir movimiento en diagonal.
- guardar mapas creados por el usuario.
- añadir más tipos de terreno.
- mejorar las animaciones.
- añadir una simulación de una ciudad.
- añadir búsqueda bidireccional.

## 15. Conclusión

Con este proyecto se ha creado una aplicación web que nos ayuda a ver diferentes algoritmos de búsqueda. Podemos jugar con la cuadrícula, colocar obstáculos, pesos y metas, ejecutar **BFS**, **DFS**, **UCS** y **A\***, ver cómo exploran todo el mapa y comparar sus resultados. Con esto se puede ver que cada algoritmo utiliza una forma diferente de buscar una ruta.

## 16. Enlaces

### GitHub

[AÑADIR AQUÍ EL ENLACE AL REPOSITORIO DE GITHUB]

### Netlify

[AÑADIR AQUÍ EL ENLACE DE LA PÁGINA PUBLICADA EN NETLIFY]